myDiL

epsi

WORKSHOP NATIONAL M2 2026-27 · GROUPE 3

PROJECT RESILIENCE-ALPHA

Cyber-résilience et reprise autonome

Dossier d'ingénierie et de gouvernance

Attaque → Détection → Remédiation → Service rétabli et prouvé

Louis DE RUDDER
M2 EISI · Infrastructure

Léo BOURLES
M2 EISI · Infrastructure

Arnaud ROYER
M2 EISI · Développement

Amile NGUYEN
M2 ECSI · Cybersécurité

Jamal INNOURI
M2 EISI · IA / Data

EPSI Montpellier · 22 octobre 2026

Données 100 % fictives · Workshop2026-M2-G3-Dossier.pdf

Synthèse

Nous avons construit une plateforme de paiement fictive, conteneurisée sur un PC Hôte, capable d'**encaisser une attaque et de se rétablir sans intervention humaine**, puis de **prouver** sa reprise par une vraie transaction. Le scénario de référence est un brute force sur la connexion, lancé depuis un PC Attaquant distinct. Les journaux d'authentification sont centralisés dans Loki ; un détecteur par apprentissage automatique (isolation forest) repère l'anomalie et transmet un incident au moteur de remédiation, qui bloque l'adresse IP au WAF, vérifie que les services répondent et exécute une transaction canari. L'incident n'est clos que si cette transaction réussit.

5,1–5,3 s
de la détection à la preuve de reprise, marge d'ingestion de 5 s incluse

≈ 4,6 s
service arrêté, redémarré par Ansible et vérifié

1,2 s
reprise du WAF après coupure (procédure P4)

P1 → P5
procédures de reprise testées, jusqu'à la reconstruction depuis Git

Les cinq piliers du sujet sont réellement interconnectés : les journaux collectés par l'infrastructure alimentent le modèle de l'équipe IA, qui déclenche le moteur de remédiation écrit par l'équipe de développement, qui agit sur le WAF de l'équipe cybersécurité ; l'analyse EBIOS RM et le PCA/PRA justifient ces choix. Nos principales limites sont assumées et documentées : un seul PC Hôte (risque R9), une attribution d'IP faussée par la virtualisation Docker (R10), un modèle validé pour le seul laboratoire et un chiffrement des secrets préparé mais non activé.

Sommaire

1 Contexte et périmètre
2 Architecture et flux
3 Rôle des composants
4 Nature de l'incident
5 Détection et reprise
6 Résultats des essais
7 Sécurité
8 Analyse des risques (EBIOS RM)
9 PCA / PRA
10 Gouvernance, NIS2/DORA et recommandations
11 Contribution de chacun
A–E Annexes

1 Contexte et périmètre

1.1 La mission

Une plateforme de paiement exploitée par une entreprise financière fictive subit des incidents répétés, aujourd'hui réparés à la main. La Direction Nationale de la Sûreté Numérique (fictive) veut vérifier, sur un périmètre maîtrisé, qu'une reprise automatique peut être conçue, observée et vérifiée. Notre démonstrateur réunit les cinq domaines demandés : développement, infrastructure, IA/data, cybersécurité et gouvernance.

1.2 Ce que nous avons choisi de protéger

Fonction	Criticité	Pourquoi
Exécution des virements (`api` , `db-paiement`)	Critique	Cœur du métier. Un virement non autorisé (intégrité) est plus grave qu'un virement retardé (disponibilité) : c'est l'événement redouté le plus grave (ER2).
Authentification (`auth` , `db-auth`)	Critique	Porte d'entrée de toute opération ; c'est aussi la cible de l'attaque démontrée.
Point d'entrée (`waf`)	Critique	Sans lui, plus aucune route n'est joignable (vérifié en §6.7) ; c'est aussi l'organe qui bloque l'attaquant.
Détection et remédiation	Élevée	Leur arrêt ne coupe pas le paiement, mais remet l'exploitation en mode manuel.
Back-office, LED d'état	Modérée	Fonctions d'administration ; leur indisponibilité est tolérable plusieurs heures.

1.3 Ce que coûte un arrêt du paiement

Chaque minute d'arrêt retarde les opérations des partenaires, dégrade la confiance des clients et mobilise le support. À titre d'ordre de grandeur, avec l'hypothèse de travail d'une plateforme traitant 50 virements par minute en pointe, une interruption de 15 minutes retarde environ 750 opérations. Au-delà de quelques heures, ou si de nombreux clients sont touchés, l'incident deviendrait vraisemblablement « majeur » au sens de DORA et déclencherait une notification réglementaire (§10.3). C'est ce raisonnement qui fixe les objectifs de reprise proposés au §9.1, et non les seules capacités techniques.

2 Architecture et flux

2.1 Trois piles Docker Compose

Pile	Rôle	Composants principaux
Infra	Réseaux partagés, observabilité, automatisation	Prometheus, node-exporter, blackbox, Loki, Promtail, Grafana, Ansible, Mosquitto (MQTT), docker-socket-proxy
Dev	Services métier, IA et leurs bases	`auth`, `api`, `remediation`, `front`, `dashboard`, `backoffice`, `face`, `detector`, `incident-preview`, `assistant`, `supervision-ai`, Mailpit, 3 bases PostgreSQL
Cyber	Le point d'entrée protégé	`waf` (Nginx + ModSecurity + règles OWASP CRS)

L'ordre de démarrage est obligatoire : infra, puis dev, puis cyber. La pile infra crée les réseaux Docker partagés que les deux autres rejoignent. Les réseaux séparent les usages : `backend` relie les applications entre elles et au WAF, `data` relie chaque application à sa base sans jamais l'exposer, `monitoring` sert à l'observabilité, `remediation` relie le moteur de remédiation et Ansible à l'accès Docker filtré. Un fichier complémentaire (`*.redondance.yml`) ajoute des instances secondaires, des répliques PostgreSQL et les sauvegardes horaires.

2.2 Vue d'ensemble

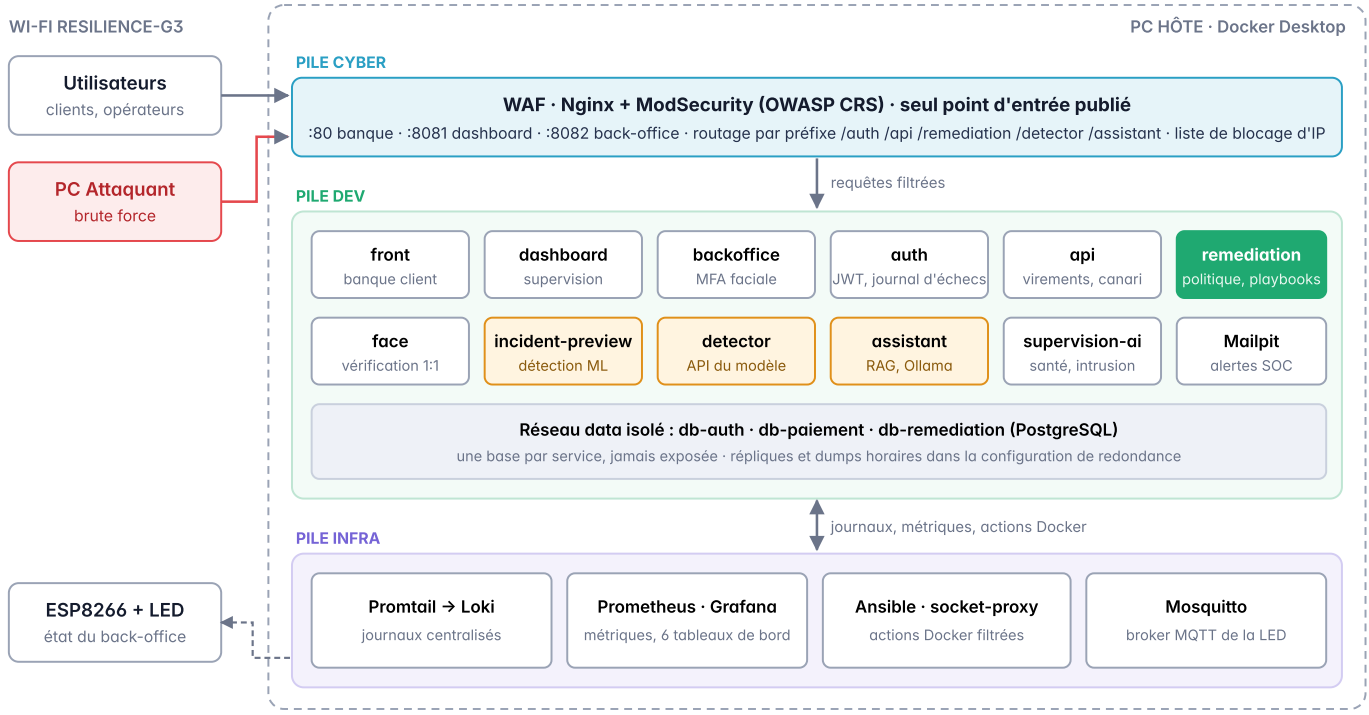


Figure 1 — Vue simplifiée de la plateforme. Le schéma détaillé de tous les flux figure en annexe A, le diagramme de séquence de l'incident en annexe B.

2.3 Flux nominal

Le WAF route par préfixe de chemin : `/auth/*`, `/api/*` et `/remediation/*` sont transmis préfixe retiré ; `/detector/*` et `/assistant/*` préfixe conservé, car ces API servent déjà ce chemin. Chaque front est compilé pour servir à la racine (`/`) : les trois applications React ne peuvent donc pas partager un seul port avec des sous-chemins sans être reconstruites, d'où un port WAF dédié par front (80, 8081, 8082). Les services s'authentifient entre eux par jetons de service (`svc:detector`, `svc:dashboard`, `svc:remediation`).

2.4 Réseau du laboratoire

Un point d'accès Wi-Fi mobile Windows dédié (`RESILIENCE-G3`, WPA2, PC Hôte en 192.168.137.1) isole le laboratoire, conformément au sujet. Le PC Hôte héberge les trois piles ; un PC Attaquant distinct s'y connecte pour lancer les scénarios offensifs ; l'ESP8266 s'y connecte pour piloter la LED. Les bases, le proxy Docker et les secrets ne sont jamais exposés. Le WAF est le point d'entrée prévu ; des ports de débogage publiés pendant le développement restent toutefois ouverts sur le PC Hôte (limite traitée en §7.7).

3 Rôle des composants

Composant	Rôle	Technologie
`front`	Banque client : connexion, virement, historique	React 19, Vite, Tailwind
`dashboard`	Supervision : état des services, incidents en temps réel, déblocage d'IP, assistant IA	React 19, Server-Sent Events
`backoffice`	Administration : connexion renforcée (visage + mot de passe), habilitations, onglet Observabilité (Grafana + Mailpit intégrés)	React 19, capture caméra
`auth`	Comptes, connexion, jetons JWT (RS256), jetons de service, journal des échecs, machine à états du back-office, publication de l'état de la LED	Node.js, Express, Prisma, PostgreSQL
`api`	Comptes, virements simulés, règle anti-fraude, transaction canari	Node.js, Express, Prisma, PostgreSQL
`remediation`	Reçoit les incidents, applique la politique, exécute les playbooks (`block_source` , `restart_service`), vérifie et prouve la reprise, notifie par e-mail	Node.js, Express, Prisma, PostgreSQL, Nodemailer
`incident-preview`	Worker de détection : lit Loki, score chaque IP avec le modèle, transmet les incidents à la remédiation (mode `deliver`)	Python, scikit-learn
`detector`	API du modèle : expose son état et les anomalies observées au dashboard	Python, FastAPI
`assistant`	Assistant documentaire RAG pour les opérateurs (§5.5)	Python, Ollama, LangChain
`supervision-ai`	Santé des services (redémarrage via Ansible) et alerte sur comportement suspect. Volontairement sans apprentissage automatique : un contrôle déterministe sur `/readyz` et sur le volume de refus WAF	Node.js, Nodemailer
`face`	Reconnaissance faciale du back-office, vérification 1:1	Python, dlib, `face_recognition` , ResNet-34
`waf`	Point d'entrée unique, routage par préfixe, pare-feu applicatif, blocage d'IP	Nginx, ModSecurity, OWASP CRS
`mailpit`	Serveur SMTP de laboratoire et interface web, reçoit les notifications d'incident	Mailpit
ESP8266 + LED	Indicateur physique de l'état d'authentification du back-office	Firmware Arduino, MQTT (Mosquitto)
Prometheus · Loki · Grafana	Métriques (services, machine, sondes), journaux centralisés, six tableaux de bord provisionnés	node-exporter, blackbox, Promtail
Ansible	Automatisation des procédures de reprise : redémarrage de service, bascule de base	Playbooks `remediation.yml` , `bascule-db.yml`

Chaque service a sa propre base PostgreSQL, dans son propre conteneur. Aucun service n'accède à la base d'un autre. `log-detector` , détecteur par seuil sans apprentissage automatique hérité d'une phase antérieure, reste dans le dépôt comme repli mais n'est pas utilisé en démonstration (§5.2).

4 Nature de l'incident

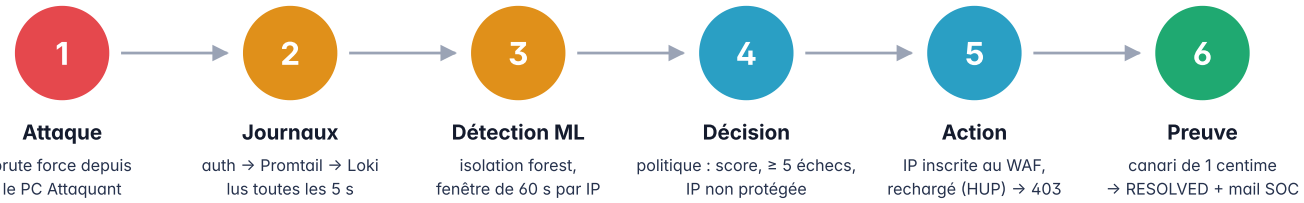
Le scénario de référence est un **brute force sur la connexion** (`POST /auth/v1/login`). Le PC Attaquant essaie un dictionnaire de mots de passe contre un compte connu (`victime@resilience.local`), volontairement doté d'un mot de passe faible pour que l'attaque puisse aboutir. Le service `auth` n'applique volontairement aucun verrouillage de compte ni limitation de débit : ce n'est pas son rôle de se défendre lui-même, c'est à la chaîne détection-remédiation de réagir. C'est ce choix qui rend la démonstration possible et mesurable ; en production, ces protections seraient ajoutées (§10.4).

Un deuxième volet porte sur la **fraude** : une fois l'accès obtenu, on tente un virement vers un IBAN externe inconnu. La règle anti-fraude de `api` le refuse indépendamment de la détection d'intrusion. Un troisième scénario, indépendant d'une attaque, couvre la **panne d'un service métier** (conteneur arrêté) et sa reprise automatique via Ansible. Nous distinguons ainsi restauration de la disponibilité (blocage, redémarrage) et protection contre la compromission (contrôle serveur du virement), qui n'appellent pas la même réponse.

Consigne additionnelle : authentification renforcée du back-office

Le back-office impose une connexion en trois étapes : identifiant, puis reconnaissance faciale, puis mot de passe. Un échec de reconnaissance ou un mot de passe erroné réinitialise le parcours ; deux échecs consécutifs bloquent l'identifiant 5 minutes, durée doublée à chaque récidive, plafonnée à 1 heure. Une LED physique (ESP8266, MQTT) reflète l'état en direct, mais elle n'est qu'un indicateur : elle ne sert jamais de preuve ni de mécanisme d'autorisation, l'accès réel dépend uniquement des contrôles côté serveur.

5 Détection et reprise



Aucune intervention humaine entre l'étape 1 et l'étape 6. Si la transaction canari échoue, l'incident reste FAILED.

Figure 2 — La boucle de reprise autonome démontrée.

5.1 Détection par apprentissage automatique

Données. Le worker `incident-preview` interroge Loki toutes les 5 secondes sur les journaux structurés de `auth` (événements `login_failed` / `login_succeeded` avec IP et compte visé). Il analyse volontairement une fenêtre décalée de 5 secondes, le temps que Loki ingère complètement les journaux. Les messages de démarrage et les lignes mal formées sont traités explicitement, pas interprétés comme de l'activité normale.

Préparation. Pour chaque IP source et chaque fenêtre glissante de 60 secondes, cinq variables sont calculées : nombre de tentatives, d'échecs, de succès, ratio d'échec et nombre de comptes distincts visés.

Modèle. Isolation forest (scikit-learn, 100 arbres), entraînée sans étiquette sur 300 fenêtres synthétiques de trafic normal : le modèle apprend ce qu'est une activité ordinaire et isole ce qui s'en écarte. Ce n'est pas une règle codée en dur : une rafale d'échecs sur plusieurs comptes, ou un ratio d'échec inhabituel, ressort même sans seuil fixé à la main.

Seuil et validation. Le seuil (0,740) est calibré sur trois fenêtres de faible activité issues d'un export réel des journaux du laboratoire (valeur maximale + 0,01), puis contrôlé sur seize autres fenêtres du même export : huit rafales d'attaque toutes détectées, huit fenêtres de faible activité ou de récupération sans alerte. Le modèle et son seuil portent alors le statut `validated`, dans la portée explicite `controlled_authentication_lab`, avec l'empreinte SHA-256 du modèle inscrite dans le fichier de seuil. Avant cette validation, un garde-fou du code (`require_validated_delivery`) refusait de démarrer en mode de livraison ; nous l'avons vérifié en le déclenchant volontairement.

Du score à l'incident. Une anomalie n'est transmise que si elle correspond à un brute force caractérisé (au moins 5 échecs et un ratio d'échec d'au moins 0,8) et ne provient pas d'une adresse de bouclage ou de passerelle partagée connue. Un seul incident est produit par poussée d'attaque : une IP signalée ne redéclenche un incident qu'après avoir cessé d'être vue comme anormale (§6.3).

Limites constatées

Seize fenêtres issues d'un seul export ne mesurent pas une performance générale : ce sont des contrôles d'acceptation du scénario, pas une vérité terrain indépendante, et le trafic normal y est peu dense. Le modèle reste entraîné sur des données synthétiques ; sa validation ne vaut que pour ce laboratoire. Le score n'est pas une probabilité d'attaque.

5.2 Configuration de démonstration

La démonstration utilise la configuration de livraison : la pile dev est lancée avec le fichier complémentaire `docker-compose.ml-delivery.yml`, qui fait passer `incident-preview` du mode `preview` (observation seule) au mode `deliver`, et `log-detector` est arrêté pour que la remédiation soit déclenchée uniquement par le modèle. Cette procédure est décrite dans `ml/ai-runtime/deploy/VALIDATED_LAB_DELIVERY.md`.

5.3 Le moteur de remédiation

`remediation` applique une politique avant d'agir. Un incident en dessous du score minimal ou du seuil d'échecs est enregistré sans action : le score seul n'est jamais pris pour une preuve. Deux playbooks :

`block_source` (intrusion)

1. valide strictement que la source est une adresse IP et qu'elle n'est pas protégée (hôte, passerelles connues) ;
2. écrit l'IP dans la liste de blocage du WAF et lui envoie un signal de rechargement (HUP) ;
3. vérifie que les services dépendants répondent (`/readyz`) ;
4. exécute une transaction canari de 1 centime, marquée synthétique, entre deux comptes internes dédiés.

`restart_service` (panne)

1. redémarre le conteneur concerné via le playbook Ansible, ou directement via le proxy Docker filtré selon le mode configuré ;
2. vérifie que le service répond de nouveau (`/readyz`) ;
3. exécute la même transaction canari de preuve.

Dans les deux cas, l'incident passe à `RESOLVED` **seulement si la transaction réussit** ; sinon il reste `FAILED` avec le motif. Une notification par e-mail part vers Mailpit avec le type d'incident, la cible, le score et le résultat. Les blocages d'IP durent 300 secondes par défaut, levés automatiquement à l'échéance ou manuellement par un opérateur depuis le dashboard.

5.4 Supervision complémentaire : panne de service et comportement suspect

- **Santé des services.** `supervision-ai` interroge `/readyz` sur `auth`, `api` et `remediation`. Au premier échec, un incident `SERVICE_DOWN` est signalé à `remediation`, qui applique `restart_service` (redémarrage Ansible, vérification, canari). Un service déjà signalé n'est pas re-signalé tant qu'il n'est pas revenu disponible.
- **Comportement suspect.** Interrogation de Loki sur les refus WAF et les échecs d'authentification ; au-delà de 15 événements sur 120 secondes, une alerte e-mail part vers le SOC, sans action automatique sur le WAF pour ne pas dupliquer le rôle du détecteur.

5.5 Assistant documentaire pour les opérateurs

L'assistant combine Ollama, exécuté sur le PC Hôte, avec LangChain. Le modèle de langage est `qwen2.5:3b` (3 milliards de paramètres, sous la limite de 8 fixée par le sujet, choisi pour tenir sur le matériel du PC Hôte) et la recherche documentaire utilise `embeddinggemma:300m`. Sa base de connaissances regroupe les documents produits par l'équipe (procédures, architecture) et le sujet, en PDF et Markdown. Chaque réponse cite ses sources ; une question sans réponse dans la base est refusée plutôt qu'inventée. L'assistant n'est accessible qu'avec un jeton `operator` ou `svc:dashboard` et ne dispose d'**aucun droit d'action** : il aide à comprendre l'incident et à retrouver la procédure, il ne se confond pas avec l'automate de remédiation.

6 Résultats des essais

Tous les essais qui suivent ont été exécutés en conditions réelles : conteneurs réels, vraies requêtes HTTP, sans simulation de code, avec horodatage et codes de réponse relevés.

6.1 Chaîne complète, automatique, sans intervention manuelle

Un brute force réel contre le compte victime a déclenché, sans aucune action manuelle, la détection par le modèle, le blocage de l'IP dans le WAF, la transaction de preuve, puis la notification e-mail. Chronologie type côté `remediation` :

Étape	Résultat
`incident_received`	reçu de `svc:detector`, type `BRUTE_FORCE`, score autour de 0,77–0,78 (seuil 0,740)
`validate_ip`	IP valide, peut être bloquée
`block_ip`	IP écrite dans la liste de blocage du WAF
`verify_ready`	services `auth` et `api` prêts
`canary_transaction`	transaction exécutée avec succès
`incident_resolved`	incident clos, e-mail envoyé via Mailpit

Avant blocage, une requête vers la banque répondait 200 ou 401 selon les identifiants. Après blocage, la même IP recevait 403 depuis le WAF : le rejet est réel au niveau du pare-feu, pas seulement une entrée en base. Délais mesurés : **5,1 à 5,3 secondes** entre la détection et la preuve de reprise. La marge d'ingestion de 5 secondes est un choix de fiabilité, pas une lenteur de la remédiation, qui agit en quelques dizaines à quelques centaines de millisecondes une fois l'incident reçu.

6.2 Déblocage manuel

Un opérateur peut lever un blocage avant son échéance depuis le dashboard (`DELETE /v1/blocks/:id` , rôle `operator` uniquement). Testé : blocage, 403 confirmé, déblocage, accès restauré (200) en moins d'une seconde, rechargement du WAF confirmé par le signal HUP reçu côté conteneur.

6.3 Re-détection après déblocage manuel : défaut trouvé et corrigé

En testant le cycle complet blocage, déblocage manuel, nouvelle attaque, la deuxième attaque, pourtant réelle, ne déclenchait plus aucun incident. Cause : dans le détecteur, chaque IP ouvrait un épisode d'anomalie persistant que rien ne refermait ; une même IP ne pouvait déclencher qu'un seul incident pour toute la durée de vie du détecteur. Le correctif ferme l'épisode dès que l'IP n'est plus vue comme anormale. Une poussée continue reste dédupliquée ; une poussée qui cesse puis reprend déclenche un nouvel incident. Vérifié en conditions réelles (blocage, déblocage, nouvelle attaque, nouveau blocage automatique) et par la suite de tests du détecteur, 36 tests et 17 sous-tests au vert.

6.4 Attaque depuis un vrai PC Attaquant

Essai réalisé avec l'outil d'attaque de l'équipe (`attacks/attacks/lab.py`), lancé depuis un poste physiquement distinct : détection, blocage WAF et preuve fonctionnent de bout en bout. Limite constatée : la virtualisation du PC Hôte (Docker Desktop, WSL2) masque l'IP réelle du poste attaquant avant qu'elle n'atteigne le WAF. L'outil dispose d'un mode documenté qui transmet l'IP explicitement, utilisé pour la démonstration ; ce contournement est traité comme un risque accepté (R10, §8.6).

6.5 Blocage du virement frauduleux après compte compromis

Connexion réelle sur le compte victime avec le mot de passe trouvé par brute force, puis tentative de virement vers un IBAN externe inconnu : **403 Forbidden**, `reason: unknown_beneficiary`, journalisé et compté (`payments_fraud_blocked_total`). La compromission du compte ne suffit pas à exécuter un virement frauduleux : le contrôle serveur est une barrière indépendante de la détection d'intrusion.

6.6 Panne de service et reprise via Ansible

Arrêt volontaire d'un conteneur (`docker stop api`) en fonctionnement normal. `supervision-ai` détecte l'absence de réponse sur `/readyz`, signale `SERVICE_DOWN` ; `remediation` redémarre le conteneur via le playbook Ansible, vérifie la disponibilité puis exécute la transaction canari. Incident résolu sans action manuelle, **MTTR d'environ 4,6 secondes**.

6.7 Perte et reprise du WAF (procédure P4)

Arrêt volontaire du conteneur `waf` : panne totale confirmée sur toutes les routes, faute de répartiteur de charge. Service rétabli en **1,2 seconde** après redémarrage, les 4 routes testées de nouveau à 200, sans perte de configuration : routage, règles ModSecurity et liste de blocage sont recréés depuis les fichiers montés.

6.8 Reconstruction complète depuis le dépôt (procédure P5)

Clonage du dépôt dans un environnement neuf et isolé, reconstruction des trois piles de zéro, création des comptes de démonstration, puis brute force réel rejoué : détection, incident, IP bloquée, transaction canari réussie, incident résolu. La chaîne autonome fonctionne à l'identique sur une reconstruction complète. Deux éléments ont dû être fournis hors dépôt Git : le modèle et son seuil (non versionnés, transférés séparément) et les secrets de configuration.

6.9 Essais restant à mener

Séquence biométrique complète en conditions de démonstration (faux rejet, verrouillage progressif) ; bascule de base (P2) rejouée et chronométrée en conditions de démonstration (mesurée en atelier à environ 2 secondes) ; tentative d'abus de l'interface de remédiation (mesure M4).

7 Sécurité

7.1 Authentification et autorisation

Mots de passe hachés avec bcrypt, jamais journalisés. Jetons JWT signés RS256, valables 15 minutes, clés publiques exposées (JWKS). Chaque virement est autorisé côté serveur à chaque appel : une connexion réussie ne suffit jamais à valider une opération financière (§6.5). Les comptes de service (`svc:detector`, `svc:dashboard`, `svc:remediation`) ont des identifiants dédiés à portée limitée : la déclaration d'un incident est réservée à l'identité `svc:detector` (utilisée par le détecteur et par `supervision-ai`), la levée d'un blocage au rôle `operator`.

7.2 Point d'entrée unique et pare-feu applicatif

Le WAF (Nginx et ModSecurity avec le jeu de règles OWASP CRS complet, plus de 850 règles actives, niveau de paranoïa 1) est la porte d'entrée publiée sur le réseau du laboratoire. Les exceptions ModSecurity sont scopées à un chemin précis (étape visage du back-office, enrôlement d'un visage, méthode `DELETE` du déblocage manuel), jamais appliquées globalement.

7.3 Privilèges du moteur de remédiation

Le moteur de remédiation dispose des droits les plus élevés de la plateforme ; nous les avons bornés à trois niveaux. Il n'accède jamais au socket Docker brut : il passe par `docker-socket-proxy`, qui n'ouvre que l'API des conteneurs (lecture, redémarrage, signal, `exec`) et ferme images, volumes, réseaux et système. Le proxy ne sait pas filtrer conteneur par conteneur : la restriction aux seuls conteneurs `api`, `waf` et `ansible` est appliquée par le code (`ALLOWED_CONTAINERS`), et ce proxy n'est joignable que depuis le réseau isolé `remediation`, partagé avec Ansible uniquement. Toutes les actions (redémarrage, signal HUP, exécution de playbook) sont journalisées. Limite assumée : `exec` et l'écriture sur l'API conteneurs, nécessaires pour lancer les playbooks, restent des privilèges puissants (§10.4).

7.4 Identification biométrique

La reconnaissance faciale est une vérification 1:1, pas une identification ouverte. Aucune image n'est conservée ni journalisée. Un échec du service de reconnaissance n'est jamais compté comme un échec de connexion.

7.5 Fonction SOC intégrée à la supervision

Un SIEM Wazuh a d'abord été déployé, puis retiré : il ne recevait aucun journal applicatif (aucun agent enregistré, le collecteur du manager ne lisant que ses propres alertes) et sa charge mémoire pesait sur le PC Hôte. Nous avons choisi une fonction SOC intégrée à la supervision existante, comme le sujet l'autorise :

- **Collecte** : Promtail lit les journaux des conteneurs `auth`, `api`, `remediation` et `waf` et les centralise dans Loki ;
- **Corrélation et lecture** : tableaux de bord Grafana *Securite-WAF-Journaux* (refus 403, volume de journaux, flux WAF en direct), *Remediation* (MTTR, incidents, journal du moteur), *Paiements-impact* et *Plateforme-santé*, intégrés au back-office dans un onglet Observabilité ;
- **Alerte** : e-mails SOC (Mailpit) à chaque incident résolu ou en échec, et sur pic d'événements suspects (§5.4).

7.6 Secrets et durcissement

Aucun secret n'est versionné : fichiers `.env` et clé privée JWT hors dépôt, vérifié par `scripts/check-secrets.sh` avant chaque rendu ; le service `auth` refuse de démarrer sans clé plutôt que d'utiliser une clé éphémère. Le chiffrement des secrets par SOPS et age est préparé (règles `.sops.yaml`, dossier `infra/secrets/`) mais **n'est pas activé** : la clé publique de l'équipe n'a pas été générée faute de temps. Côté durcissement, les services applicatifs (`auth`, `api`, `remediation`, `supervision-ai`, services IA) tournent sous un utilisateur non privilégié, les conteneurs IA ont un système de fichiers en lecture seule et aucune base n'est publiée.

7.7 Limites assumées

Limite	Correction prévue
Ports de débogage encore publiés sur le PC Hôte (`auth` 3010, `api` 3001, `remediation` 3002, `detector` 8011, `assistant` 8012, fronts 5173–5175, Prometheus 9090) : ils contournent le WAF, même si les API restent protégées par JWT	Les retirer de la configuration ou les filtrer au pare-feu Windows
Chiffrement SOPS/age préparé mais non activé ; clé privée JWT non sauvegardée hors du PC Hôte	Activer SOPS ; clé JWT dans un coffre (Vault, KMS)
Service de reconnaissance faciale sans authentification sur ses routes internes (réseau interne uniquement)	Jeton de service sur `face`
Grafana accessible en lecture anonyme (nécessaire à son intégration dans le back-office) et publié sur le port 3000	Authentification par proxy, accès via le WAF uniquement
Broker MQTT en connexions anonymes	Comptes MQTT et ACL par sujet
Périmètre de confiance IP du WAF et de `auth` élargi aux plages privées pour contourner le masquage d'IP (§6.4)	Resserrer avant tout déploiement réel
Modèle de détection validé pour le scénario du laboratoire uniquement	Réentraînement et suivi sur trafic réel

8 Analyse des risques (EBIOS RM)

Analyse structurée suivant la trame EBIOS Risk Manager (5 ateliers), adaptée au périmètre du laboratoire : plateforme de paiement fictive, données synthétiques, trois piles Docker Compose sur un seul PC Hôte. Les cotations sont des échelles de travail propres au projet (gravité de 1 à 4), pas une échelle officielle de l'ANSSI, et aucune conformité normative n'est revendiquée.

8.1 Valeurs métier et biens supports (atelier 1)

Valeur métier	Besoin de sécurité	Biens supports
Authentification	Confidentialité des identifiants, accès réservé aux utilisateurs autorisés	`auth`, `db-auth`, front, WAF, clé JWT
Transactions fictives	Intégrité et autorisation de chaque opération	`api`, `db-paiement`, WAF
Disponibilité du service	Maintien ou rétablissement des fonctions critiques	WAF, `auth`, `api`, bases et répliques
Continuité et réponse à incident	Détecter, contenir, vérifier, reprendre	détecteur ML, `supervision-ai`, `remediation`, Loki, Prometheus, Grafana, Ansible, sauvegardes
Administration	Accès réservé aux personnes autorisées	back-office, reconnaissance faciale, mot de passe, MQTT, ESP8266

8.2 Événements redoutés

ID	Événement redouté	Gravité
ER2	Virement fictif non autorisé exécuté	4 · critique
ER5	Contournement de l'identification biométrique du back-office	4 · critique
ER7	Modification non autorisée des habilitations	4 · critique
ER1	Accès frauduleux à un compte utilisateur	3 · majeur
ER4	Journaux absents ou altérés	3 · majeur
ER3	Indisponibilité de l'authentification ou du service	2 · modéré
ER6	Rejet biométrique injustifié d'un utilisateur autorisé	2 · modéré
ER8	État de LED trompeur ou MQTT détourné (indicateur uniquement)	2 · modéré

8.3 Sources de risque (atelier 2)

Hypothèses de travail, pas des acteurs observés : un attaquant externe cherchant à compromettre un compte et détourner un virement (ER1, ER2) ; un attaquant cherchant à saturer ou contourner l'authentification (ER3) ; un attaquant ciblant les mécanismes de sécurité pour empêcher ou détourner la remédiation (ER1–ER4) ; un attaquant ciblant le back-office (ER5–ER7) ; un attaquant ciblant les communications de contrôle de la LED (ER8).

8.4 Scénarios stratégiques (atelier 3)

S1, compromission de compte et tentative de fraude : brute force sur la connexion, accès au compte si les contrôles échouent, puis tentative de virement. C'est le scénario démontré (§4, §6). Quatre autres scénarios sont retenus : dégradation de l'authentification par un volume élevé de tentatives (S2, ER3) ; défaillance ou détournement de la remédiation elle-même (S3, ER1–ER4) ; accès non autorisé au back-office (S4, ER5, ER7) ; indication de sécurité trompeuse via la LED ou MQTT (S5, ER8).

8.5 Risques et traitement (ateliers 4 et 5)

ID	Risque	Gravité	Mesures de traitement	Statut et preuve
R1	Compromission de compte puis tentative de virement (ER1, ER2)	4	M1 détection ML branchée sur la remédiation · M2 contrôle serveur de chaque virement · M3 blocage temporaire de l'IP au WAF	Testé §6.1, §6.5
R2	Tentatives répétées perturbant l'authentification (ER3)	2	M1, M3 · M6 surveillance de disponibilité (Prometheus, Grafana)	Testé §6.1–6.3
R3	Échec, détournement ou excès de la remédiation (ER1–ER4)	3	M4 interface protégée (jetons de service, rôles, liste blanche de conteneurs) · M7 statut `FAILED` et alerte si la preuve échoue	En place non éprouvé par une attaque dédiée
R4	Journaux absents ou altérés pendant l'incident (ER4)	3	M5 centralisation dans Loki · alertes e-mail SOC	En place rétention non sauvegardée
R5	Contournement de l'identification faciale (ER5)	4	M9 séquence en trois étapes, verrouillage progressif, vérification serveur	En place essai complet restant
R6	Faux rejet biométrique d'un utilisateur autorisé (ER6)	2	M9 · échec du service jamais compté comme échec de connexion	En place taux non mesuré

ID	Risque	Gravité	Mesures de traitement	Statut et preuve
R7	Modification non autorisée des habilitations (ER7)	4	M10 vérification des rôles côté serveur	Dans le code non testé
R8	Manipulation de la LED ou des messages MQTT (ER8)	2	M11 LED séparée de toute autorisation	Par construction
R9	Perte du PC Hôte et de l'environnement local (ER3, ER4)	4	M8 bascule, restauration et reconstruction (P2, P3, P5)	Testé §6.8, sauvegardes locales

8.6 Risques résiduels

R9, perte du PC Hôte, reste le risque résiduel principal : la reconstruction depuis Git est démontrée, mais toutes les piles et toutes les sauvegardes résident sur une seule machine. **R10, usurpation d'IP interne au réseau du laboratoire**, découle du contournement nécessaire à l'attribution d'IP (§6.4) : accepté pour la durée du workshop, à corriger avant tout déploiement réel. **R3** reste sensible tant que les privilèges du proxy Docker ne sont pas réduits (§7.3).

9 PCA / PRA

9.1 Objectifs de reprise proposés et valeurs mesurées

Les objectifs ci-dessous sont **proposés au commanditaire** à partir de l'impact métier (§1.3) ; ils restent à valider par le métier. Les valeurs mesurées décrivent le laboratoire : elles montrent la marge disponible, elles ne remplacent pas un objectif. PTMA : période maximale tolérable d'interruption ; RTO : délai cible de reprise ; RPO : perte de données maximale tolérable.

Fonction	PTMA	RTO	RPO	Mesuré au laboratoire
Paiement (`api` , `db-paiement`)	30 min	5 min	aucun virement validé perdu (réplique) ; ≤ 1 h sur dump	service : ≈ 4,6 s · bascule de base : ≈ 2 s
Authentification (`auth` , `db-auth`)	30 min	5 min	quasi nul (réplique) ; ≤ 1 h sur dump	bascule de base : ≈ 2 s
Point d'entrée (`waf`)	15 min	1 min	sans objet (configuration versionnée)	1,2 s
Détection et remédiation	4 h	30 min	≤ 1 h (historique des incidents)	reconstruite avec la plateforme (P5)
PC Hôte complet	1 jour	4 h	≤ 1 h	reconstruction depuis Git réussie, durée non chronométrée

Pendant une perte de la détection, le service de paiement continue mais l'exploitation repasse en surveillance manuelle renforcée sur Grafana : c'est le mode dégradé prévu. Les sauvegardes sont des dumps horaires avec rotation sur 24 fichiers.

9.2 Organisation de crise

Quatre rôles : **chef de crise** (qualifie l'incident, autorise la sortie de crise), **pilote technique** (coordonne bascule, restauration ou reconstruction), **support technique** (exécute les procédures, collecte les preuves), **communication** (diffuse l'alerte et les mises à jour, prépare une éventuelle notification réglementaire). Escalade en trois étapes : détection et qualification, décision et action (choix de la procédure P1 à P5), vérification et clôture. Les incidents gérés automatiquement (§5) ne mobilisent la cellule que si la preuve de reprise échoue (`FAILED`).

9.3 Procédures

Proc.	Scénario	Statut
P1	Perte d'une instance applicative (`api` , `auth` ...)	Testé alias réseau vers l'instance secondaire pour `api` ; redémarrage via `supervision-ai` + Ansible pour tout service surveillé (§6.6)
P2	Perte d'une base primaire, bascule de réplique	En place playbook Ansible `bascule-db.yml` , bascule mesurée à environ 2 s
P3	Restauration depuis un dump horaire	En place commande fournie et testée pour la base paiement
P4	Perte du WAF	Testé redémarrage, service rétabli en 1,2 s (§6.7)
P5	Perte du PC Hôte	Testé reconstruction complète depuis Git (§6.8)

9.4 Risques qui subsistent après la reprise

Limite	Priorité
PC Hôte unique, aucune sauvegarde hors site	P1
Clé JWT non sauvegardée : sa perte invalide toutes les sessions et impose une régénération	P1
Périmètre de confiance réseau élargi pour la démonstration (§7.7)	P1
Un blocage d'IP traite la disponibilité, pas une compromission déjà réussie : un compte dont le mot de passe a été trouvé doit encore être réinitialisé	P1
Reconnaissance faciale non redondée	P2

10 Gouvernance, NIS2/DORA et recommandations

10.1 Organisation et processus

Cinq pôles (développement, infrastructure, cybersécurité, IA/data, IoT), une contribution identifiée par personne (§11) et des décisions d'architecture prises en commun. Dépôt Git unique en monorepo (`services/`, `packages/`, `infra/`, `ml/`, `docs/`, `attacks/`, `iot/`), un fichier `docker-compose.<pile>.yml` par pile, Conventional Commits vérifiés par un hook Git, contrats d'API OpenAPI. Revue croisée des livraisons inter-pôles : chaque livraison a été testée en conditions réelles avant d'être considérée acquise, jamais validée sur la seule base d'une déclaration. Suivi sur un tableau Trello partagé (annexe C).

10.2 Décisions et compromis retenus

- **Brancher le ML seulement une fois validé.** Le détecteur n'a été branché sur la livraison automatique qu'après validation sur du trafic réel du laboratoire, sur une portée documentée. L'équipe a d'abord refusé de contourner le garde-fou par une modification directe des fichiers, le temps que le pôle IA/data mène sa propre évaluation.
- **Fiabilité plutôt que vitesse.** La marge d'ingestion de 5 secondes rallonge la détection mais évite de décider sur des journaux incomplets.
- **Blocage temporaire plutôt que définitif.** 300 secondes suffisent à couper une rafale sans bannir durablement un utilisateur légitime partageant la même IP.
- **Preuve métier plutôt que santé technique.** Un conteneur redémarré n'est pas une reprise : seule une transaction réussie clôt l'incident.
- **Fonction SOC intégrée plutôt que SIEM dédié.** Wazuh n'apportait aucun journal applicatif et coûtait en mémoire ; Loki et Grafana couvrent le besoin du démonstrateur.
- **Transparence sur les écarts.** Le contournement d'attribution d'IP et le chiffrement des secrets non activé sont documentés comme risques acceptés plutôt que passés sous silence. Le défaut de re-détection (§6.3) a été corrigé et revérifié avant mise en service.

10.3 NIS2 et DORA

Une entreprise financière réelle opérant cette plateforme relèverait de DORA (règlement (UE) 2022/2554), qui s'applique en priorité à NIS2 (directive (UE) 2022/2555) pour le secteur financier. Notre laboratoire fictif est hors champ : nous utilisons ces textes comme grille de lecture, en distinguant ce que le prototype démontre de ce qu'il ne peut pas démontrer.

Exigence	Ce que le démonstrateur prouve	Hors de portée d'un prototype de 4 jours
Gestion du risque TIC DORA ch. II · NIS2 art. 21 a	Analyse EBIOS RM reliée à l'architecture réelle, mesures M1–M11 suivies avec leur preuve	Cadre de gestion approuvé et revu par l'organe de direction
Gestion des incidents DORA ch. III · NIS2 art. 21 b	Détection, classification (`BRUTE_FORCE`, `SERVICE_DOWN`), réponse automatique, traçabilité dans Loki et la base `remediation`, alerte SOC	Classification « majeur » et notification aux autorités : sous 4 h après classification et au plus tard 24 h après détection (DORA) ; alerte précoce sous 24 h et notification sous 72 h (NIS2, art. 23)
Continuité, sauvegardes, crise DORA art. 11–12 · NIS2 art. 21 c	Procédures P1–P5 testées, répliques, dumps horaires, organisation de crise	Site de secours, sauvegardes hors site, exercices de crise réguliers
Tests de résilience DORA ch. IV	Injection de pannes (§6.6–6.8) et attaque Red Team réelle depuis un poste distinct	Tests de pénétration fondés sur la menace (TLPT) menés par des tiers

Exigence	Ce que le démonstrateur prouve	Hors de portée d'un prototype de 4 jours
Contrôle d'accès, MFA, cryptographie NIS2 art. 21 h, i, j	JWT RS256, bcrypt, MFA faciale du back-office, rôles côté serveur	Secrets chiffrés au repos (SOPS non activé), gestion des clés en coffre
Prestataires TIC tiers DORA ch. V · NIS2 art. 21 d	Dépendances identifiées : images Docker publiques, modèles Ollama, règles OWASP CRS	Registre des prestataires, clauses contractuelles, stratégie de sortie

10.4 Recommandations à la direction et ce que nous ferions en contexte industriel

1. **Sortir du point unique de défaillance (R9).** Deux sites ou zones, un orchestrateur gérant nativement redémarrage et répartition de charge, sauvegardes chiffrées hors site. Aucun RTO crédible n'est tenable sur une seule machine.
2. **Défendre au plus tôt.** Limitation de débit et verrouillage progressif dans `auth`, MFA pour les clients, WAF en niveau de paranoïa 2 après ajustement des faux positifs. La remédiation automatique devient alors un second rideau, pas le seul.
3. **Fiabiliser l'attribution d'IP (R10).** WAF en frontal réseau réel, en-têtes de proxy acceptés uniquement depuis des relais de confiance.
4. **Industrialiser le ML.** Réentraînement sur trafic réel, suivi des faux positifs, revue humaine des changements de seuil (la traçabilité par empreinte est déjà en place).
5. **Réduire les privilèges de la remédiation (R3).** Remplacer l'accès au proxy Docker par une API d'actions fermée ou un opérateur à droits minimaux, fermer les ports de débogage.
6. **Protéger les secrets.** Activer SOPS ou Vault, clé JWT en coffre avec rotation.
7. **Gouverner.** Faire valider PTMA et RTO par le métier, écrire la procédure de notification NIS2/DORA, planifier des exercices de chaos réguliers.

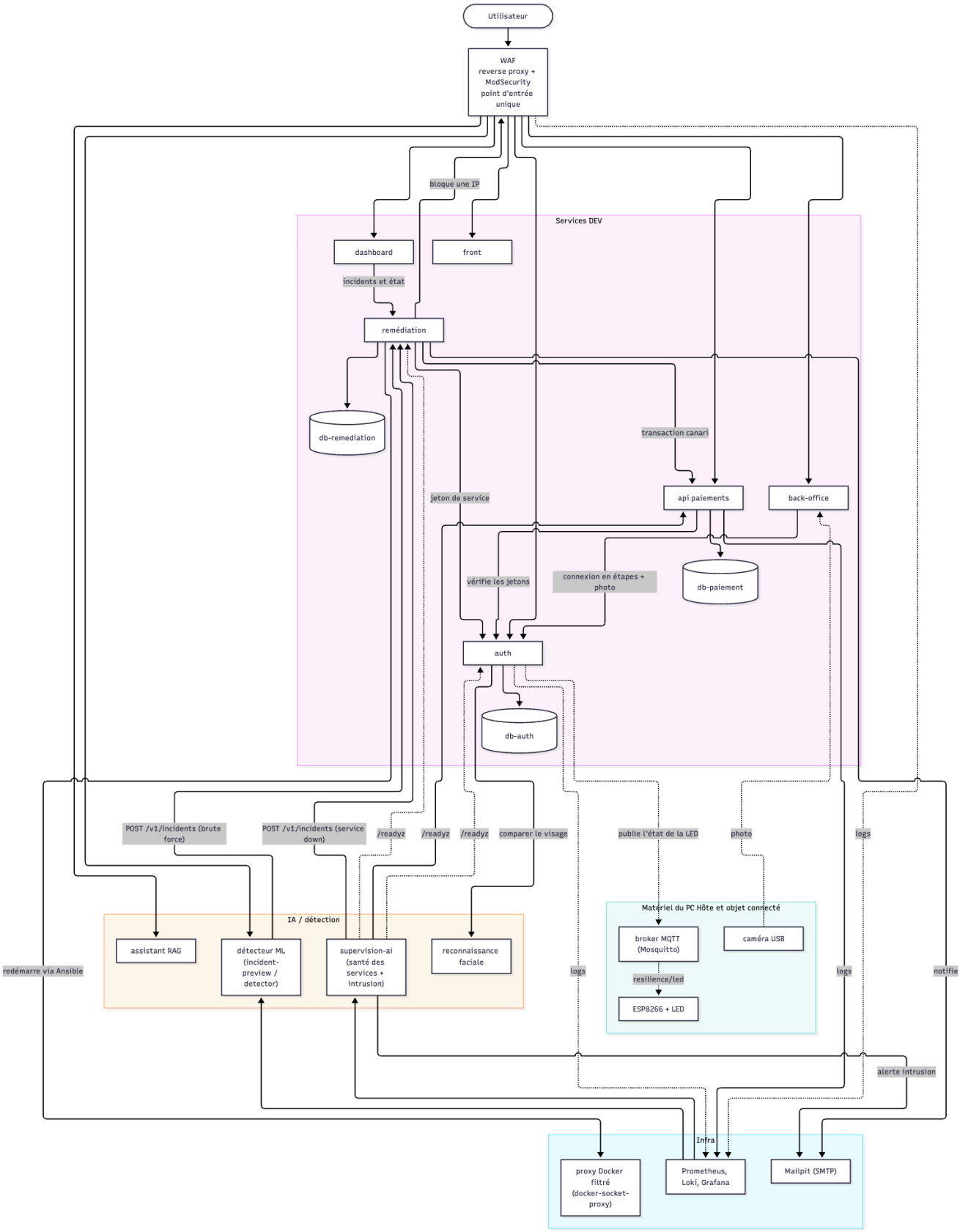
Ce que nous ferions différemment : fixer les objectifs métier et la stratégie de secrets dès le premier jour, et brancher le modèle plus tôt sur des journaux réels plutôt que synthétiques, pour laisser plus de temps à sa validation.

11 Contribution de chacun

Membre	Pôle	Réalisations principales
Arnaud ROYER M2 EISI	Développement	Les sept services métier (`auth` , `api` , `remediation` , `front` , `dashboard` , `backoffice`) et le package partagé ; moteur de remédiation et ses playbooks ; service de reconnaissance faciale ; intégration Docker Compose de la pile applicative ; notification e-mail (Mailpit) ; correctif de re-détection du détecteur ; corrections de bout en bout
Léo BOURLES M2 EISI	Infrastructure, IoT	Piles d'infrastructure et d'observabilité (Prometheus, Loki, Grafana et tableaux de bord) ; configuration du WAF et blocage automatique d'IP ; playbook Ansible de remédiation ; `supervision-ai` et `log-detector` ; firmware ESP8266, machine à états de la LED, intégration MQTT
Louis DE RUDDER M2 EISI	Infrastructure	Images Docker de l'infrastructure ; découpage du Compose en trois piles ; redondance (instances secondaires, répliques PostgreSQL, sauvegardes horaires) et bascule de base
Amile NGUYEN M2 ECSI	Cybersécurité, gouvernance	Scripts d'attaque Red Team ; déploiement puis retrait argumenté du SIEM Wazuh ; analyse EBIOS RM ; PCA/PRA ; mesures de reprise (MTTR) lors des tests de panne
Jamal INNOURI M2 EISI	IA / Data	Détecteur par apprentissage automatique (variables, modèle, calibrage, validation sur trafic réel), worker de livraison des incidents vers la remédiation, assistant documentaire RAG (Ollama, LangChain) ; pull requests #1 à #5 (annexe D)

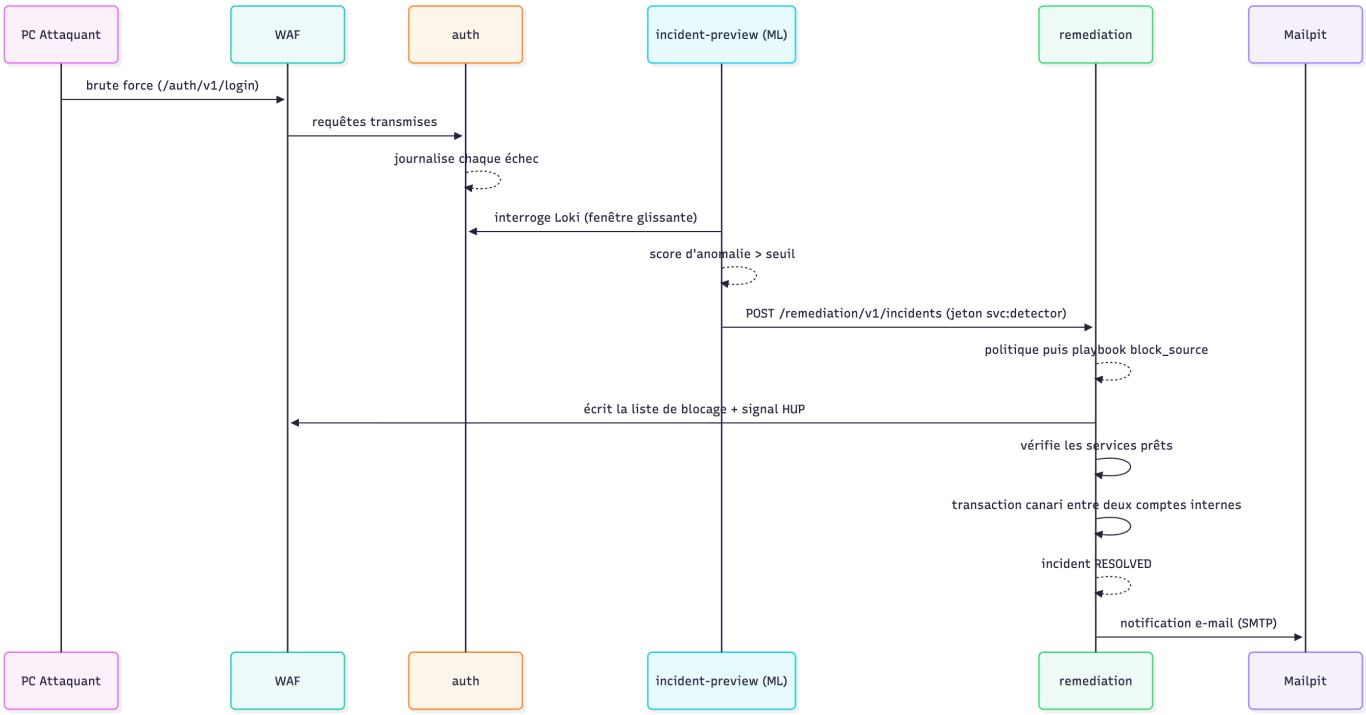
Contributions établies à partir de l'historique Git du dépôt et des livraisons de chaque pôle. Le tableau Trello et les statistiques GitHub figurent en annexes C et D.

Annexe A — Vue d'ensemble détaillée des flux

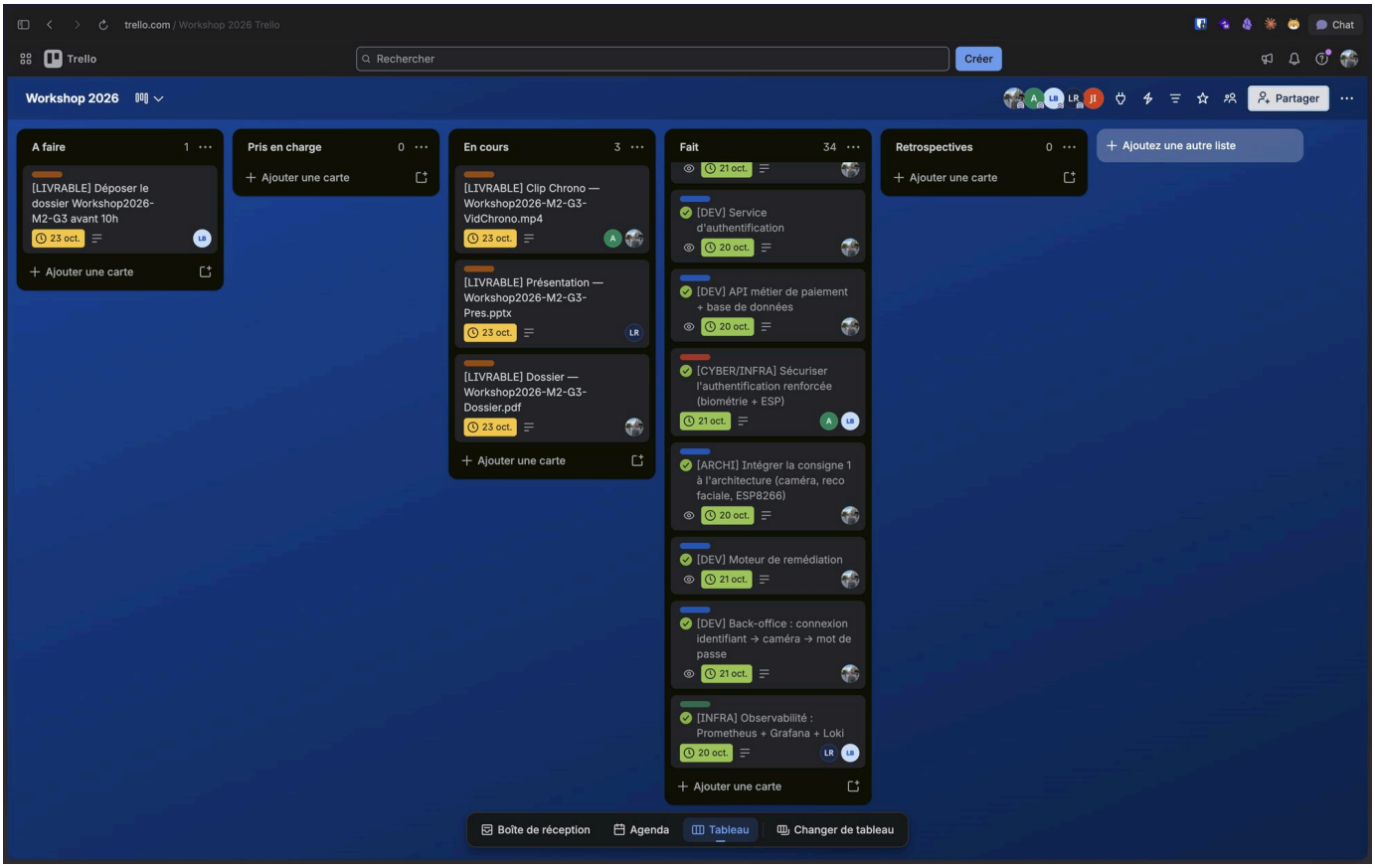


Traits pleins : appels directs. Pointillés : vérification ou observation.

Annexe B — Séquence de l'incident démontré

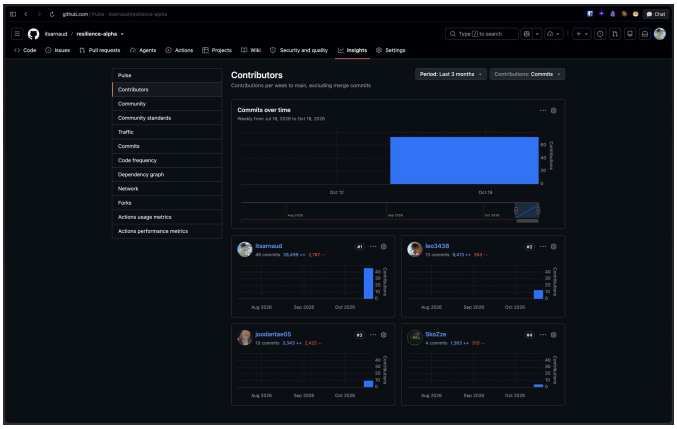


Annexe C — Suivi du projet (Trello)

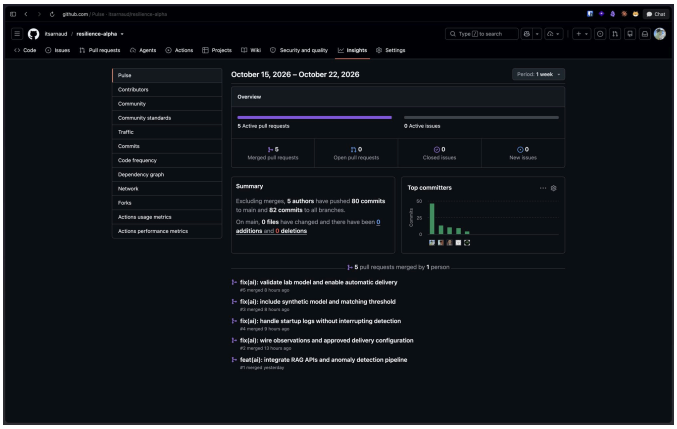


Le tableau ne reflète pas l'intégralité des tâches : certaines actions rapides ou corrections ponctuelles n'y ont pas été créées en carte. Il reste la source de suivi principale pour les livrables et les chantiers structurants.

Annexe D — Contribution Git



Contributeurs du dépôt.



Activité de la semaine (Pulse).

Le graphique des contributeurs ne fait pas apparaître Jamal : ses commits ont été poussés depuis un compte Git local non lié à son compte GitHub. Son travail reste visible dans l'historique (auteur `J.innouri`) et dans les pull requests qu'il a ouvertes, toutes fusionnées dans `main` :

PR	Titre	Fusionnée le
#1	`feat(ai)` : intégration des API RAG et du pipeline de détection d'anomalies	21/10
#2	`fix(ai)` : branchement des observations et de la configuration de livraison approuvée	22/10
#3	`fix(ai)` : inclusion du modèle synthétique et du seuil correspondant	22/10
#4	`fix(ai)` : traitement des journaux de démarrage sans interrompre la détection	22/10
#5	`fix(ai)` : validation du modèle pour le laboratoire et activation de la livraison automatique	22/10

C'est la PR #5 qui lève le garde-fou de livraison automatique décrit en §5.1, après évaluation sur le trafic réel du laboratoire.

Annexe E — Documents de référence dans le dépôt

Fichier	Contenu
`README.md`	Installation, lancement des trois piles, vérification
`docs/architecture.md`	Architecture, flux, sécurité
`docs/api/*.openapi.yaml`	Contrats d'API : auth, paiements, remédiation, IA
`ml/ai-runtime/deploy/VALIDATED_LAB_DELIVERY.md`	Validation du modèle et activation de la livraison
`ml/ai-runtime/reports/validation/`	Rapports de validation du détecteur
`infra/compose/ansible/playbooks/`	Playbooks de redémarrage et de bascule de base
`attacks/`	Scripts d'attaque Red Team, à lancer depuis le PC Attaquant